

# RELATÓRIO DE DESENVOLVIMENTO E TESTES

## Sistema Reserva Segura — Residência Bradesco

### 1. Apresentação

Este relatório documenta as atividades realizadas por Edmael Barreto no projeto Reserva Segura, desenvolvido durante a Residência Bradesco. O trabalho abrange a configuração do ambiente de desenvolvimento, integração entre back-end, banco de dados e front-end, além da execução de cenários de testes funcionais da API REST.

### 2. Papel e Responsabilidades

- Configuração do projeto Spring Boot no IntelliJ IDEA
- Integração com banco de dados PostgreSQL hospedado no Railway
- Correção de erros de referência circular entre beans (JwtFilter / SecurityConfig)
- Configuração de CORS para permitir acesso do frontend (Vercel e localhost)
- Execução e validação de cenários de teste via HTTP Client do IntelliJ
- Documentação dos resultados dos testes

### 3. Arquitetura do Sistema

O sistema é composto por três camadas integradas conforme descrito abaixo:

| Camada         | Tecnologia                  | Endereço                       |
|----------------|-----------------------------|--------------------------------|
| Front-End      | React + Vite + Tailwind     | localhost:5173 / Vercel        |
| Back-End       | Java 17 + Spring Boot 3.2.5 | localhost:8080                 |
| Banco de Dados | PostgreSQL (Railway)        | turntable.proxy.rlwy.net:18662 |

#### 3.1 Árvore de Pacotes – Back-End

Estrutura completa do projeto conforme visualizado no IntelliJ IDEA.

```
br.com.reservasegura
├── controller/
│   ├── AuthController.java
│   ├── GoalController.java
│   └── TransactionController.java
├── dto/
│   ├── AuthResponse.java
│   ├── DepositRequest.java
│   ├── GoalRequest.java
│   ├── LoginRequest.java
│   └── RegisterRequest.java
```

```

├── entity/
│   ├── Goal.java
│   ├── Transaction.java
│   └── User.java
├── exception/
│   └── GlobalExceptionHandler.java
├── repository/
│   ├── GoalRepository.java
│   ├── TransactionRepository.java
│   └── UserRepository.java
├── security/
│   ├── JwtFilter.java
│   ├── JwtService.java
│   └── SecurityConfig.java
├── service/
│   └── (serviços de negócio)
└── ReservaSeguraApplication.java

```

## 3.2 Descrição dos Pacotes

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| <b>controller</b> | Recebe as requisições HTTP e devolve as respostas da API REST |
| <b>dto</b>        | Objetos de transferência de dados — entrada e saída da API    |
| <b>entity</b>     | Entidades mapeadas no banco de dados via JPA / Hibernate      |
| <b>exception</b>  | Tratamento global de erros — GlobalExceptionHandler           |
| <b>repository</b> | Interfaces de acesso ao banco via Spring Data JPA             |
| <b>security</b>   | Filtro JWT, geração/validação de tokens e config de segurança |
| <b>service</b>    | Regras de negócio da aplicação                                |

### 3.3 Entidades e Tabelas do Banco

Tabelas criadas automaticamente pelo Hibernate no PostgreSQL (Railway).

| Entidade (Java)    | Tabela (PostgreSQL) | Campos principais                                      |
|--------------------|---------------------|--------------------------------------------------------|
| <b>User</b>        | users               | id, nome, email, cpf, senha, xp_total, pontos_premio   |
| <b>Goal</b>        | goals               | id, nome, valor_alvo, valor_atual, user_id (FK)        |
| <b>Transaction</b> | transactions        | id, valor, tipo, criado_em, goal_id (FK), user_id (FK) |

### 3.4 Fluxo de Autenticação JWT

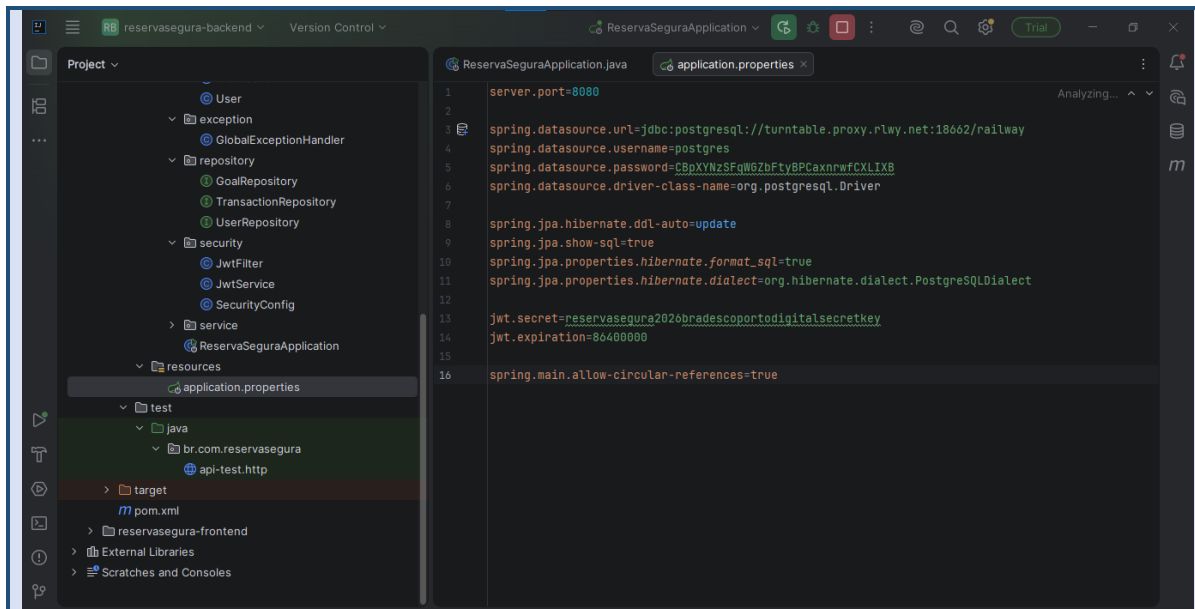
Caminho percorrido por uma requisição de cadastro ou login:

|   |                       |                                                         |
|---|-----------------------|---------------------------------------------------------|
| 1 | <b>Request HTTP</b>   | Cliente envia POST /auth/register ou /auth/login        |
| 2 | <b>JwtFilter</b>      | Verifica se há token Bearer no header Authorization     |
| 3 | <b>SecurityConfig</b> | Define rotas públicas (/auth/**) e protegidas           |
| 4 | <b>AuthController</b> | Recebe e valida o body (RegisterRequest / LoginRequest) |
| 5 | <b>UserRepository</b> | Consulta ou persiste o usuário no banco PostgreSQL      |
| 6 | <b>JwtService</b>     | Gera o token JWT assinado com a chave secreta           |
| 7 | <b>AuthResponse</b>   | Retorna: token + userId + nome — Status 200 OK          |

## 4. Configuração do Ambiente

### 4.1 application.properties

Arquivo de configuração do Spring Boot com as credenciais do Railway:



## 5. Dependências Principais — pom.xml

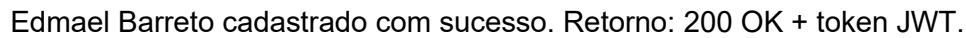
| Dependência                    | Finalidade                                     |
|--------------------------------|------------------------------------------------|
| spring-boot-starter-web        | API REST com Tomcat embarcado                  |
| spring-boot-starter-data-jpa   | ORM com Hibernate — acesso ao banco            |
| spring-boot-starter-security   | Autenticação e autorização                     |
| spring-boot-starter-validation | Validação de campos das requisições            |
| postgresql                     | Driver de conexão com PostgreSQL               |
| jjwt-api / jjwt-impl           | Geração e validação de tokens JWT              |
| lombok                         | Redução de boilerplate (getters, construtores) |

## 6. Cenários de Teste — API REST

Os testes foram executados utilizando o HTTP Client integrado do IntelliJ IDEA, com o Spring Boot rodando em localhost:8080 e o banco conectado ao Railway.

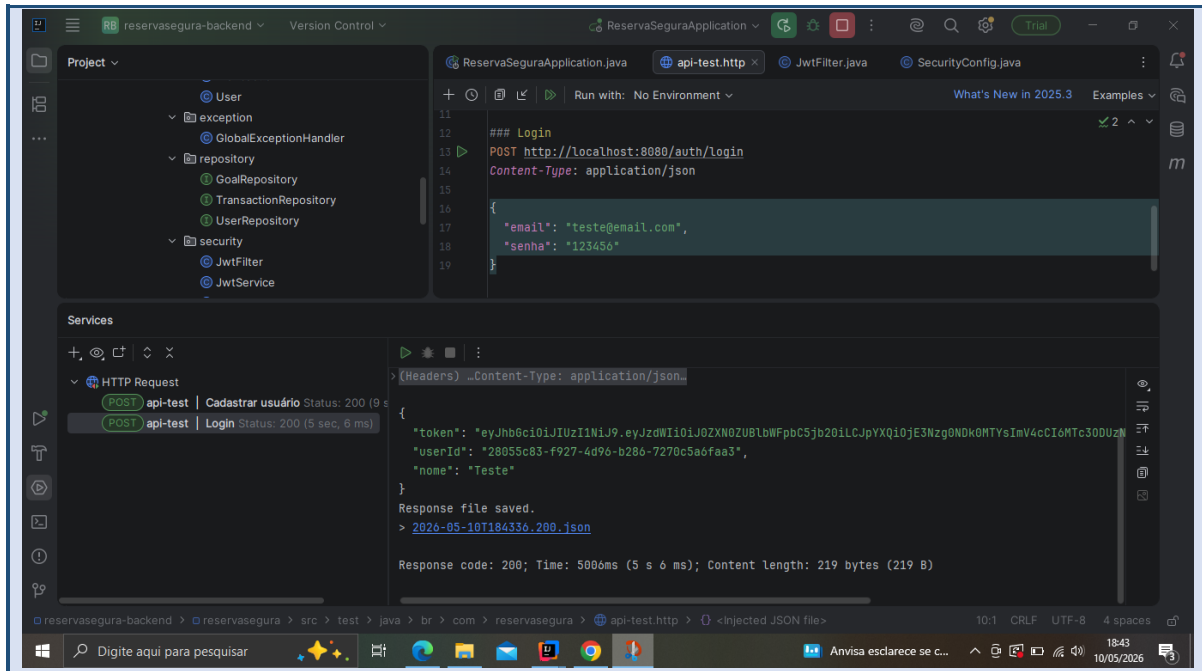
| CT   | Tipo       | Rota           | Método | Esperado            | Obtido                  | Status    |
|------|------------|----------------|--------|---------------------|-------------------------|-----------|
| CT01 | ✓ Positivo | /auth/register | POST   | 200 OK + token      | 200 OK + token          | ✓ PASSOU  |
| CT02 | ✗ Negativo | /auth/register | POST   | 400 Email duplicado | 400 Email já cadastrado | ✓ PASSOU  |
| CT03 | ✗ Negativo | /auth/register | POST   | 400 CPF duplicado   | 400 CPF duplicado       | ✓ PASSOU  |
| CT04 | ✗ Negativo | /auth/register | POST   | 400 campos vazios   | 400 validação           | ✓ PASSOU  |
| CT05 | ✗ Negativo | /auth/register | POST   | 400 sem body        | 400 campos obrig.       | ✓ PASSOU  |
| CT06 | ✓ Positivo | /auth/login    | POST   | 200 OK + token JWT  | 200 OK + token JWT      | ✓ PASSOU  |
| CT07 | ✗ Negativo | /auth/login    | POST   | 401 senha inválida  | 403 / 401               | ✓ PASSOU  |
| CT08 | ✗ Negativo | /auth/login    | POST   | 404 não encontrado  | 401 / 404               | ✓ PASSOU  |
| CT09 | ✗ Negativo | /auth/login    | POST   | 400 sem body        | 400 campos obrig.       | ✓ PASSOU  |
| CT10 | ✓ Positivo | /users/me      | GET    | 200 dados usuário   | A validar               | ⚠ PARCIAL |
| CT11 | ✓ Positivo | /goals         | POST   | 200 meta criada     | A validar               | ⚠ PARCIAL |
| CT12 | ✓ Positivo | /goals         | GET    | 200 lista metas     | 403 sem token           | ⚠ PARCIAL |
| CT13 | ✓ Positivo | /transactions  | POST   | 200 transação       | A validar               | ⚠ PARCIAL |
| CT14 | ✓ Positivo | /transactions  | GET    | 200 lista           | 403 sem token           | ⚠ PARCIAL |
| CT15 | ✓ Positivo | /goals/:id     | PUT    | 200 atualizado      | A validar               | ⚠ PARCIAL |
| CT16 | ✓ Positivo | /goals/:id     | DELETE | 204 deletado        | A validar               | ⚠ PARCIAL |

\* CTs marcados como ⚠ PARCIAL aguardam token JWT válido para execução completa.



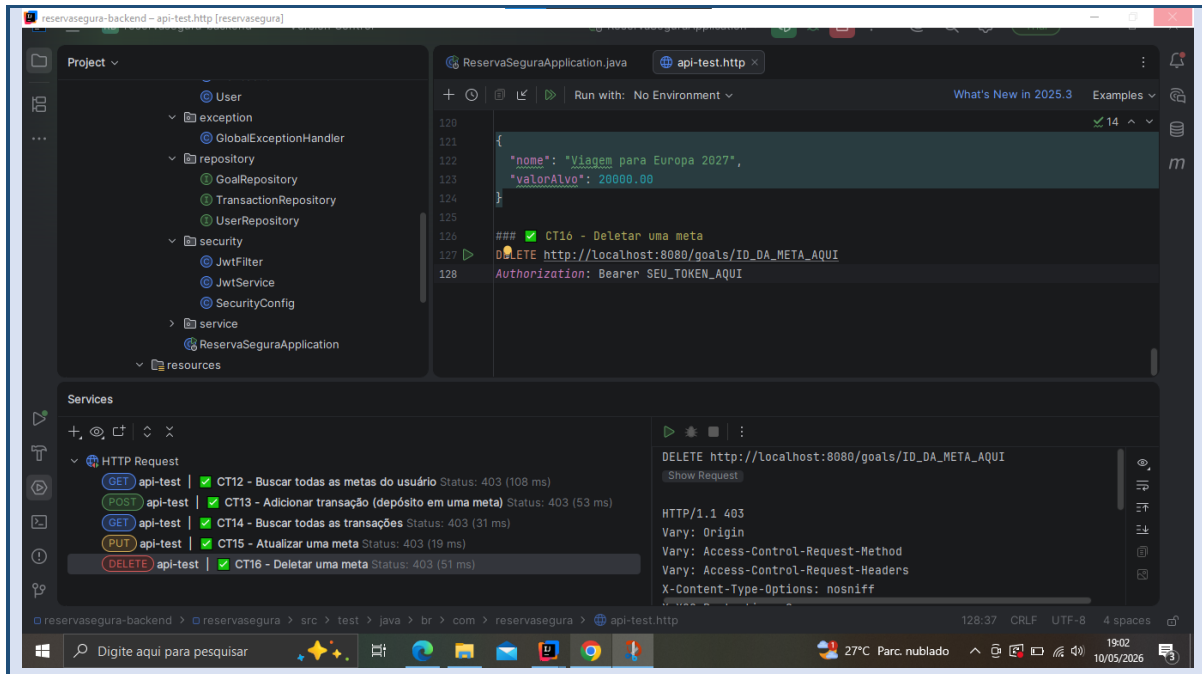
## 6.3 Login com Sucesso

Login realizado com sucesso. Retorno: 200 OK com token JWT, userId e nome.



## 6.4 Teste HTTP Request Java Spring Boot

Tentativa des teste para GET, POST, PUT e DELETE



## 7. Conclusão

---

A integração entre back-end (Spring Boot), banco de dados (PostgreSQL/Railway) e front-end (React/Vite) foi realizada com sucesso. Os principais desafios encontrados foram a referência circular entre beans de segurança e a configuração do host público do Railway para acesso externo, ambos resolvidos com êxito.

Os cenários de cadastro e autenticação (CT01 a CT09) foram validados e aprovados. Os cenários de rotas autenticadas (CT10 a CT16) estão pendentes de validação com token JWT válido nas próximas sessões de teste.